Web-Page Loading Using requests, BeautifulSoup and LangChain Core

1. Today's Master Formula

Web Page

↓

requests

↓

HTML

↓

BeautifulSoup

↓

Extracted Text

↓

LangChain Document

Short form:

requests → HTML → BeautifulSoup → Extracted Text → Document

2. Why Do We Need Web-Page Loading?

A web page may contain many different parts:

- Headers
- Navigation menus
- Sidebars
- Main content
- Footers
- Advertisements

But in many applications, we may not want every piece of information.

We may require only:

Relevant Content

For example:

Complete Web Page

↓

Header

Navigation

Sidebar

Main Content

Footer

Advertisements

↓

Select Only Required Content

↓

Create Document

3. Main Requirement

Suppose useful information is available on a website.

We want to:

1. Fetch the webpage

↓

2. Get its HTML

↓

3. Parse the HTML

↓

4. Find relevant content

↓

5. Extract readable text

↓

6. Create LangChain Document

4. What Is Beautiful Soup?

Definition

Beautiful Soup is a Python library used for parsing HTML content.

For today's lesson, we use Beautiful Soup mainly to:

Parse HTML

↓

Search HTML Elements

↓

Extract Readable Text

Installation:

python -m pip install beautifulsoup4

Important import:

from bs4 import BeautifulSoup

5. Important Clarification

BeautifulSoup is **not** a LangChain class.

Its responsibility is:

HTML Parsing

LangChain's:

Document

is used later to represent the extracted information.

So:

BeautifulSoup

↓

Parse HTML

↓

Extract Text

and:

LangChain Document

↓

Represent Extracted Information

6. What Is the requests Module?

To fetch an accessible public webpage, we can use:

requests

The requests library sends an HTTP request to a web server and retrieves the response.

Easy classroom analogy:

Our Python Program

↓

requests

↓

Courier Boy

↓

Web Server

↓

HTML Response

So we can think:

requests acts like a courier boy between our Python program and the web server.

7. Important Limitation of requests

An HTTP request is **not** a JavaScript-enabled browser.

Some websites may require:

- Login
- JavaScript execution
- Browser interaction
- Special authentication
- Other specialised tools

So:

Static / Accessible HTML

↓

requests

↓

Usually Works

But:

JavaScript-Generated Content

↓

requests

↓

May Not Contain Final Visible Content

8. Understanding BeautifulSoup()

Consider:

```
soup = BeautifulSoup(
   html,
   "html.parser"
)
```

Purpose

Convert HTML text into a structured object so that we can search and extract information easily.

9. Understanding the Arguments

```
BeautifulSoup(
    html,
    "html.parser"
)
```

BeautifulSoup

A class from the bs4 library used to parse HTML.

html

A Python string containing HTML content.

"html.parser"

Python's built-in HTML parser.

soup

The resulting BeautifulSoup object representing the parsed HTML document.

Flow:

HTML String

↓

BeautifulSoup()

↓

Parsed HTML Structure

↓

soup

10. Why Parse HTML?

Suppose we have:

```
<html>
  <body>
    <main>
      <h1>Python Training</h1>
    </main>
  </body>
</html>
```

Initially, this is just HTML text.

After:

```
soup = BeautifulSoup(
  html,
  "html.parser"
)
```

we can search the structure using methods such as:

select_one()

select()

get_text()

11. Understanding select_one()

Consider:

```
main = soup.select_one(
    "main"
)
```

Purpose

Find the first <main> HTML element in the parsed document.

12. Understanding Each Part

soup.select_one("main")

soup

The parsed HTML document.

select_one()

A Beautiful Soup method that finds the **first matching element** using a CSS selector.

"main"

A CSS selector looking for:

<main>

main

Contains the matching HTML element as a Beautiful Soup Tag object.

13. Important Behaviour

If <main> exists:

main = soup.select_one("main")

returns:

Tag Object

If <main> does not exist:

None

So:

Matching Element Found?

/ \

Yes No

↓ ↓

Tag None

14. select_one() vs select()

Method	Purpose	Return Type
select_one("main")	Finds first matching element	Tag or None
select("main")	Finds all matching elements	ResultSet / list-like collection

Easy memory:

select_one()

↓

ONE

select()

↓

ALL

15. Final Learning Formula

BeautifulSoup()

↓

Convert HTML into

Searchable Structure

select_one()

↓

Find First Matching Element

get_text()

↓

Extract Readable Text

16. Understanding get_text()

Consider this HTML:

```
<main>
   <h1>Python Training</h1>
   <p>Learn Python</p>
   <p>Learn LangChain</p>
</main>
```

Suppose:

```
main = soup.select_one(
   "main"
)
```

Now we can extract text using:

```
main.get_text()
```

17. Example 1 — get_text() Without Arguments

```
print(
    main.get_text()
)
```

Possible output:

```
Python Training
Learn Python
Learn LangChain
```

The exact whitespace may depend on the HTML formatting.

18. Example 2 — get_text(" ")

```
print(
    main.get_text(" ")
)
```

Possible output:

```
Python Training Learn Python Learn LangChain
```

Here:

" "

acts as the separator between extracted text pieces.

19. Example 3 — get_text(" ", strip=True)

```
main.get_text(
    " ",
    strip=True
)
```

This means:

Extract Text

+

Join Using One Space

+

Remove Extra Surrounding Whitespace

This is very useful for creating clean page_content.

20. Understanding strip=True

Suppose HTML contains:

```
<p>    Learn Python    </p>
```

Without stripping:

```
"    Learn Python    "
```

With:

strip=True

we get:

"Learn Python"

21. Program 1 — Creating a Document From a Local HTML Page

Objective

Create a local HTML file, parse it using Beautiful Soup, select the main content, and create a LangChain Document.

```
from pathlib import Path


from bs4 import BeautifulSoup


from langchain_core.documents import Document



# ========================================

# PREPARE SAMPLE HTML PAGE

# ========================================


file_path = (

    Path(__file__).resolve().parent
```

```
    / "course_page.html"
)


file_path.write_text(
    """
    <html lang="en">

    <head>
        <title>Course Updates</title>
    </head>

    <body>

        <main>

            <h1>Python Training</h1>

            <p>
                New batch starts on Monday.
```

```
        </p>

      </main>

   </body>

   </html>
   """,
   encoding="utf-8"
)


# ========================================
# PARSE HTML
# ========================================

html = file_path.read_text(
   encoding="utf-8"
)
```

```
soup = BeautifulSoup(
    html,
    "html.parser"
)


main = soup.select_one(
    "main"
)


# ========================================
# CREATE LANGCHAIN DOCUMENT
# ========================================

document = Document(
    page_content=main.get_text(
        " ",
        strip=True
```

```
    ),

    metadata={
        "source":
            file_path.as_uri(),

        "source_type":
            "html",

        "title":
            soup.title.get_text(
                strip=True
            )
    }
)



# ========================================
# DISPLAY RESULT
# ========================================
```

```
print(
    "TITLE:",
    document.metadata["title"]
)


print(
    "CONTENT:",
    document.page_content
)


print(
    "SOURCE TYPE:",
    document.metadata["source_type"]
)


print(
```

```
    "SOURCE:",
    document.metadata["source"]
)
```

22. Program 1 Flow

course_page.html

↓

read_text()

↓

HTML String

↓

BeautifulSoup

↓

Parsed HTML

↓

select_one("main")

↓

Main Content

↓

get_text()

↓

Readable Text

↓

Document

23. Understanding file_path.as_uri()

We store:

"source": file_path.as_uri()

This converts the local file path into a URI.

Conceptually:

Local File Path

↓

as_uri()

↓

File URI

Example:

file:///C:/project/course_page.html

24. Extracting the Page Title

We use:

```
soup.title.get_text(
   strip=True
)
```

Suppose HTML contains:

```
<title>Course Updates</title>
```

Then:

```
soup.title.get_text(
   strip=True
)
```

returns:

Course Updates

This is stored in:

```
document.metadata["title"]
```

25. Program 1 Document Structure

Document

```
  |
  ├── page_content
  |     ↓
  |   Python Training
  |   New batch starts on Monday.
  |
  └── metadata
        |
        ├── source
        ├── source_type
        └── title
```

26. Program 2 — Loading a Public Web Page

Required Installation

python -m pip install requests beautifulsoup4

27. Objective

Fetch an accessible public webpage using requests, parse it using Beautiful Soup, extract useful content, and create a LangChain Document.

```
import requests


from bs4 import BeautifulSoup


from langchain_core.documents import Document



# ========================================
# APPROVED DEMO URL
# ========================================

url = (
    "https://www.durgasoftonline.com/"
    "s/pages/new-batches"
)
```

```
# ========================================
# FETCH AND PARSE THE WEB PAGE
# ========================================

try:

    response = requests.get(
        url,

        headers={
            "User-Agent":
            "LangChainClassroomDemo/1.0"
        },

        timeout=15
    )


    response.raise_for_status()
```

```
soup = BeautifulSoup(
    response.text,
    "html.parser"
)


title = (
    soup.title.get_text(
        strip=True
    )

    if soup.title

    else "Not available"
)


main = soup.select_one(
    "main"
```

```
)


content_node = (
    main
    if main
    else soup.body
)


content = content_node.get_text(
    " ",
    strip=True
)


# ====================================
# CREATE LANGCHAIN DOCUMENT
# ====================================
```

```
document = Document(
    page_content=content,

    metadata={
        "source":
            url,

        "source_type":
            "web",

        "title":
            title
    }
)



# ====================================
# DISPLAY RESULT
# ====================================
```

```
print(
    "TOTAL DOCUMENTS: 1"
)


print(
    "TITLE:",
    document.metadata["title"]
)


print(
    "SOURCE:",
    document.metadata["source"]
)


print(
    "CONTENT PREVIEW:",
    document.page_content[:250]
```

```
    )


except requests.RequestException as error:

    print(
        "WEB PAGE COULD NOT BE LOADED"
    )


    print(
        "ERROR:",
        type(error).__name__
    )
```

28. Program 2 Complete Flow

URL

↓

requests.get()

↓

HTTP Response

↓

response.text

↓

HTML

↓

BeautifulSoup

↓

Parsed HTML

↓

select_one("main")

↓

Relevant Content

↓

get_text()

↓

Document

29. Understanding requests.get()

```
response = requests.get(
    url,
    ...
)
```

sends an HTTP GET request.

Simple:

Python

↓

requests.get()

↓

Web Server

↓

Response

30. Understanding response

The returned object:

response

contains information from the web server.

Important examples:

response.text

gives the response body as text.

For an HTML page:

response.text

↓

HTML String

31. Understanding User-Agent

We use:

```
headers={
   "User-Agent":
      "LangChainClassroomDemo/1.0"
}
```

The User-Agent header tells the web server which client/application is making the request.

Conceptually:

Python Program

↓

HTTP Request

↓

User-Agent

↓

Web Server

32. Understanding timeout=15

timeout=15

means:

Do not allow the request operation to wait indefinitely.

Conceptually:

Send Request

↓

Wait for Response

↓

Timeout Protection

33. Understanding response.raise_for_status()

response.raise_for_status()

checks the HTTP response status.

Conceptually:

HTTP Response

↓

Successful?

/ \

Yes No

↓ ↓

Continue Raise HTTPError

If the response indicates success:

Program Continues

If an HTTP error occurs:

requests.exceptions.HTTPError

is raised.

34. Understanding response.text

We pass:

response.text

to:

```
BeautifulSoup(
   response.text,
   "html.parser"
)
```

So:

Web Server

↓

HTTP Response

↓

response.text

↓

HTML String

↓

BeautifulSoup

35. Safe Title Extraction

We use:

```
title = (
    soup.title.get_text(
        strip=True
    )

    if soup.title
```

```
    else "Not available"
)
```

This means:

Does <title> Exist?

```
     / \
    Yes  No
     ↓   ↓
Extract   "Not available"
Title
```

36. Selecting the Main Content

We use:

```
main = soup.select_one(
    "main"
)
```

Then:

```
content_node = (
    main
    if main
    else soup.body
```

)

Meaning:

<main> Exists?

/ \

Yes No

↓ ↓

Use Use

main body

37. Why Prefer <main>?

A webpage may contain:

Navigation

Header

Sidebar

Main Content

Footer

Advertisements

Normally, the <main> element is intended for the primary page content.

So:

Whole Web Page

↓

select_one("main")

↓

Relevant Main Content

38. Extracting Clean Text

```
content = content_node.get_text(
    " ",
    strip=True
)
```

This:

Removes HTML Tags

↓

Extracts Text

↓

Uses Space Separator

↓

Strips Extra Whitespace

39. Why Show Only 250 Characters?

We use:

document.page_content[:250]

This means:

Complete Page Content

↓

[:250]

↓

First 250 Characters

Useful when webpage content is very large.

40. Error Handling

We use:

except requests.RequestException as error:

This catches request-related problems.

Possible examples:

Connection Error

Timeout

HTTP Error

Network Problem

Then:

type(error).__name__

shows the error class name.

41. Program 3 — Custom Web Page Loader Using BaseLoader

Objective

Create our own reusable LangChain-compatible webpage loader using:

requests

+

BeautifulSoup

+

Document

+

BaseLoader

42. Complete Program

import requests

from bs4 import BeautifulSoup

```
from langchain_core.documents import Document

from langchain_core.document_loaders import (
    BaseLoader
)



# ========================================
# CUSTOM WEB PAGE LOADER
# ========================================

class CustomWebLoader(
    BaseLoader
):

    def __init__(
        self,
        url
    ):
```

```
    self.url = url


def lazy_load(self):

    response = requests.get(
        self.url,

        headers={
            "User-Agent":
                "LangChainClassroomDemo/1.0"
        },

        timeout=15
    )


    response.raise_for_status()
```

```
soup = BeautifulSoup(
    response.text,
    "html.parser"
)


title = (
    soup.title.get_text(
        strip=True
    )

    if soup.title

    else "Not available"
)


main = soup.select_one(
    "main"
```

```
)


content_node = (
    main
    if main
    else soup.body
)


if content_node is None:

    content_node = soup


content = content_node.get_text(
    " ",
    strip=True
)
```

```
        yield Document(
            page_content=content,

            metadata={
                "source":
                    self.url,

                "source_type":
                    "web",

                "title":
                    title
            }
        )



# ========================================
# SPECIFY WEB PAGE
# ========================================
```

```
url = (
    "https://www.durgasoftonline.com/"
    "s/pages/new-batches"
)



# ========================================
# LOAD WEB PAGE DOCUMENTS
# ========================================

loader = CustomWebLoader(
    url
)



try:

    documents = loader.load()
```

```
# ====================================
# DISPLAY RESULTS
# ====================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


for document in documents:


    print(
        "\nTITLE:"
    )

    print(
        document.metadata["title"]
```

```
)


print(
   "\nSOURCE:"
)

print(
   document.metadata["source"]
)


print(
   "\nCONTENT:"
)

print(
   document.page_content
)
```

```
except requests.RequestException as error:

    print(
        "WEB PAGE COULD NOT BE LOADED"
    )



    print(
        "ERROR:",
        type(error).__name__
    )
```

43. Understanding class CustomWebLoader(BaseLoader)

```
class CustomWebLoader(
    BaseLoader
):
```

means:

CustomWebLoader

↓

Inherits From

↓

BaseLoader

Our custom class follows the LangChain loader interface.

44. Understanding __init__(self, url)

```
def __init__(
    self,
    url
):

    self.url = url
```

This stores the webpage URL inside:

self.url

So it can later be used inside:

lazy_load()

45. Understanding lazy_load()

def lazy_load(self):

contains the actual webpage-loading logic.

Its job is:

Fetch Page

↓

Parse HTML

↓

Find Relevant Content

↓

Extract Text

↓

yield Document

46. Why Use yield Document(...)?

We use:

yield Document(...)

instead of:

return Document(...)

because lazy_load() follows the loader design where Documents are yielded.

Conceptually:

lazy_load()

↓

yield Document

47. What Does loader.load() Do?

Later we write:

documents = loader.load()

We do **not** need to override load() in this classroom implementation.

The inherited loader interface collects the Documents produced through:

lazy_load()

into a list.

Concept:

lazy_load()

↓

yield Document

↓

BaseLoader.load()

↓

List of Documents

48. Why Add Another Fallback?

Program 3 uses:

```
content_node = (
    main
    if main
    else soup.body
)
```

Then:

```
if content_node is None:

    content_node = soup
```

So the fallback logic becomes:

Use <main>

↓

If Missing

↓

Use <body>

↓

If Missing

↓

Use Entire Parsed Document

49. Content Selection Logic

Parsed HTML

↓

Is <main> Present?

/ \

Yes No

↓ ↓

main body

↓

Is body Present?

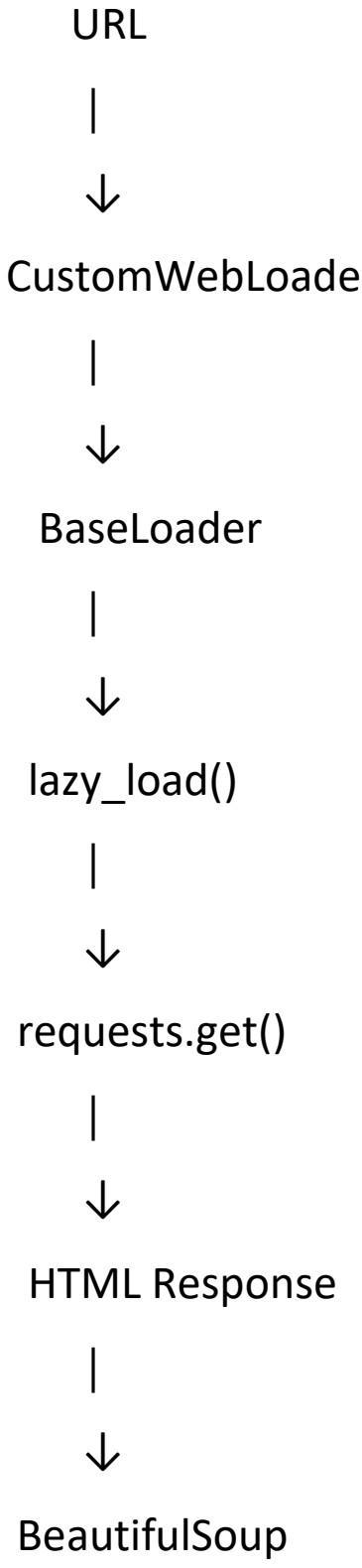
/ \

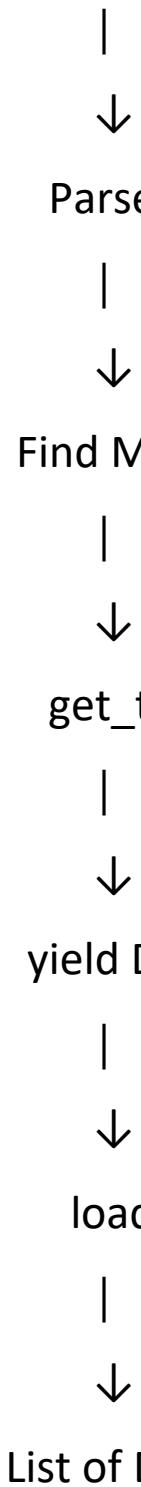
Yes No

↓ ↓

body soup

50. Custom Loader Architecture




Parse HTML
Find Main Content
get_text()
yield Document
load()
List of Documents

51. Responsibilities of Each Component

requests

Fetch Webpage

BeautifulSoup

Parse HTML

+

Search Elements

+

Extract Text

Document

Represent Extracted Information

BaseLoader

Provide Standard Loader Interface

Easy memory:

requests

↓

FETCH

BeautifulSoup

↓

PARSE

Document

↓

REPRESENT

BaseLoader

↓

STANDARDIZE

52. Local HTML vs Public Web Page

Local HTML

HTML File

↓

read_text()

↓

BeautifulSoup

Public Web Page

URL

↓

requests.get()

↓

response.text

↓

BeautifulSoup

The main difference is how we obtain the HTML.

53. Complete Comparison

LOCAL HTML

==========

File

↓

read_text()

↓

HTML

↓

BeautifulSoup

↓

Document

PUBLIC WEB PAGE

==========

URL

↓

requests

↓

HTML

↓

BeautifulSoup

↓

Document

54. Common Mistake — Thinking BeautifulSoup Downloads Websites

No.

Beautiful Soup mainly parses HTML.

It does not itself perform the HTTP request in these examples.

requests

↓

Get HTML

BeautifulSoup

↓

Parse HTML

55. Common Mistake — Thinking requests Parses HTML

No.

requests retrieves the response.

Beautiful Soup parses the HTML.

requests

↓

FETCH

BeautifulSoup

↓

PARSE

56. Common Mistake — Thinking select_one() Returns a List

No.

soup.select_one("main")

returns:

Tag

or:

None

For all matches, use:

soup.select("main")

57. Common Mistake — Ignoring None

This is unsafe:

```
main = soup.select_one(
    "main"
)

content = main.get_text()
```

because if <main> does not exist:

main = None

Then calling:

main.get_text()

will fail.

That is why Program 2 uses fallback logic.

58. Common Mistake — Extracting the Entire Page Without Filtering

If we directly extract all text:

Header

Navigation

Main Content

Footer

Advertisements

may all become part of the Document.

Instead, we may choose:

```
soup.select_one(
   "main"
)
```

to focus on relevant content.

59. Common Mistake — Thinking get_text() Removes All Unwanted Content Automatically

get_text() extracts text from the selected HTML node.

So the quality of the result depends on:

Which HTML Node

We Selected

If we select:

Whole Page

we may get lots of extra text.

If we select:

<main>

we may get more relevant text.

60. Common Mistake — Thinking requests Executes JavaScript

No.

A normal HTTP request does not behave like a full browser.

So:

JavaScript-Rendered Page

↓

requests

↓

May Receive Only Initial HTML

61. Common Mistake — Ignoring HTTP Errors

Always remember:

response.raise_for_status()

This helps detect unsuccessful HTTP status responses.

62. Three Programs — Quick Revision

Program 1

Local HTML File

↓

BeautifulSoup

↓

Document

Program 2

Public URL

↓

requests

↓

BeautifulSoup

↓

Document

Program 3

Public URL

↓

CustomWebLoader

↓

BaseLoader

↓

requests

↓

BeautifulSoup

↓

yield Document

63. Important Code Revision

Fetch Webpage

```
response = requests.get(
    url,
    timeout=15
)
```

Check HTTP Status

```
response.raise_for_status()
```

Parse HTML

```
soup = BeautifulSoup(
    response.text,
    "html.parser"
)
```

Find Main Content

```
main = soup.select_one(
    "main"
)
```

Extract Text

```
content = main.get_text(
    " ",
    strip=True
)
```

Create Document

```
Document(
    page_content=content,
    metadata={
        "source": url
```

```
    }
)
```

64. Important Learning Points

1. requests can retrieve accessible webpage HTML.
2. BeautifulSoup parses HTML.
3. BeautifulSoup is not a LangChain class.
4. Document represents the extracted information.
5. BeautifulSoup(html, "html.parser") converts HTML text into a parsed structure.
6. select_one() returns the first matching element or None.
7. select() returns all matching elements.
8. get_text() extracts readable text.
9. " " in get_text(" ") acts as a text separator.
10. strip=True removes unnecessary surrounding whitespace.
11. <main> can be selected to focus on primary webpage content.
12. requests.get() sends an HTTP GET request.
13. response.text gives the returned HTML text.

14. raise_for_status() checks HTTP status and raises an HTTP-related error for unsuccessful responses.
15. timeout prevents unlimited waiting.
16. A normal HTTP request does not execute JavaScript.
17. BaseLoader can be used to create a reusable custom webpage loader.
18. lazy_load() yields Documents.
19. The inherited load() collects yielded Documents into a list.
20. Webpage loading logic should include suitable fallback and error handling.

65. Today's Master Architecture

WEB PAGE

|

↓

URL

|

↓

requests

|

↓

HTTP Response

|

↓

response.text

|

↓

HTML

|

↓

BeautifulSoup

|

↓

Parse HTML

|

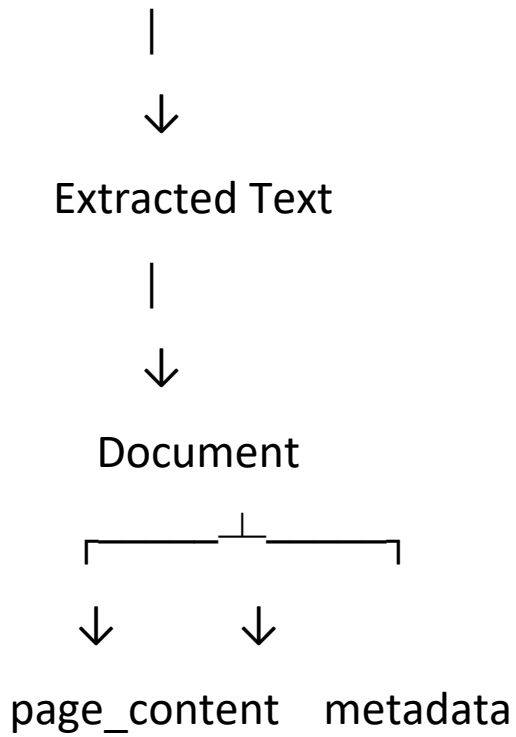
↓

Select Content

|

↓

get_text()



66. Custom Loader Master Formula

URL

↓

CustomWebLoader

↓

BaseLoader

↓

lazy_load()

↓

requests

↓

BeautifulSoup

↓

Extract Text

↓

yield Document

↓

load()

↓

List of Documents

67. Final Memory Formula

requests

↓

FETCH

BeautifulSoup

↓

PARSE

select_one()

↓

SELECT

get_text()

↓

EXTRACT

Document

↓

REPRESENT

BaseLoader

↓

STANDARDIZE

68. One-Line Conclusion

Use requests to fetch webpage HTML, BeautifulSoup to parse and select relevant HTML content, get_text() to extract readable text, LangChain Document to represent that information, and BaseLoader when we want the logic to become a reusable LangChain-compatible webpage loader.